



Bootstrapping a cluster with kubeadm init and join



Tim Warner

Principal Author | IT Ops | Pluralsight

TechTrainerTim.com



Four framing questions



What does kubeadm init actually create under the hood?



Which kubeadm init flags are critical for the CKA exam?



How does the bootstrap token secure the node join process?



How do I set up kubectl access after initializing the control plane?





“Our three VMs are prepped and containerd is running. Time to bootstrap the cluster. Get the pod-network-cidr flag right on init, or we'll be tearing it down and starting over. The CKA exam gives you one shot at this with a timer running.”



**What does kubeadm init actually create
under the hood?**



What kubeadm init creates



Self-signed CA in
/etc/kubernetes/pki/ signs
all cluster certificates



Certificates for API server,
etcd, scheduler, controller
manager, and kubelet
client



Static pod manifests in
/etc/kubernetes/manifests
/ for control plane
components



kubelet monitors manifests
directory and runs control
plane as static pods



admin.conf kubeconfig file
for cluster administrator
access



Bootstrap token and join
command for adding
worker nodes



Inside kubeadm init: eight phases you can run solo



Each phase is independently runnable: `kubeadm init phase <name>`



**Which kubeadm init flags are critical for
the CKA exam?**



kubeadm init flags that matter on exam day

`--pod-network-cidr`
192.168.0.0/16 (Tigera
Calico default — must
match)

`--apiserver-advertise-`
`address:` control plane
IP workers will connect
to

`--kubernetes-version:`
pins the cluster to the
exam-specified version

`--control-plane-`
`endpoint:` required for
HA clusters — not
needed for single-node
control plane

Mismatch = pods get
IPs but cannot route —
silent timeout



Configuration over flags: the declarative kubeadm init workflow

Generate the default init config (declarative -- the v1.35 pro move)

```
kubeadm config print init-defaults > init.yaml
```

Edit init.yaml -- four changes that scale to HA later

```
# networking.podSubnet: 192.168.0.0/16          (matches Calico)
```

```
# localAPIEndpoint.advertiseAddress: 192.168.56.10
```

```
# controlPlaneEndpoint: k8s.globomantics.local:6443    (HA-ready, see next slide)
```

```
# apiServer.certSANs: [k8s.globomantics.local, 192.168.56.10, 10.0.0.1]
```

Bootstrap with the config -- --upload-certs prepares for HA join

```
sudo kubeadm init --config init.yaml --upload-certs
```

Configure kubectl access (same as flag-based flow)

```
mkdir -p $HOME/.kube
```

```
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

```
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Workers join with the command from init output (or regenerate -- next slide)

```
sudo kubeadm join k8s.globomantics.local:6443 --token <t> --discovery-token-ca-cert-hash sha256:<h>
```



Future-proof for HA: the `--control-plane-endpoint` hook

Today: single control plane. Tomorrow: three CPs behind a load balancer.

The trap if you SKIP this flag:

API server cert SAN list contains only the node IP. Adding a load balancer later means certs no longer match -- you must run `kubeadm certs renew + reissue` every `kubeconfig`.

The fix (one line in `init.yaml` -- 10 seconds today saves 30 minutes later):

```
controlPlaneEndpoint: k8s.globomantics.local:6443
```

How it scales:

Today -> `k8s.globomantics.local` resolves to `192.168.56.10` (single CP)

Course 03 -> same DNS resolves to HAProxy fronting 3 CPs (existing certs still valid)

CKA blueprint objective: Implement and configure a highly-available control plane



**How does the bootstrap token secure
the node join process?**



Bootstrap tokens and kubeadm join workflow

kubeadm init generates a token with 24-hour default TTL

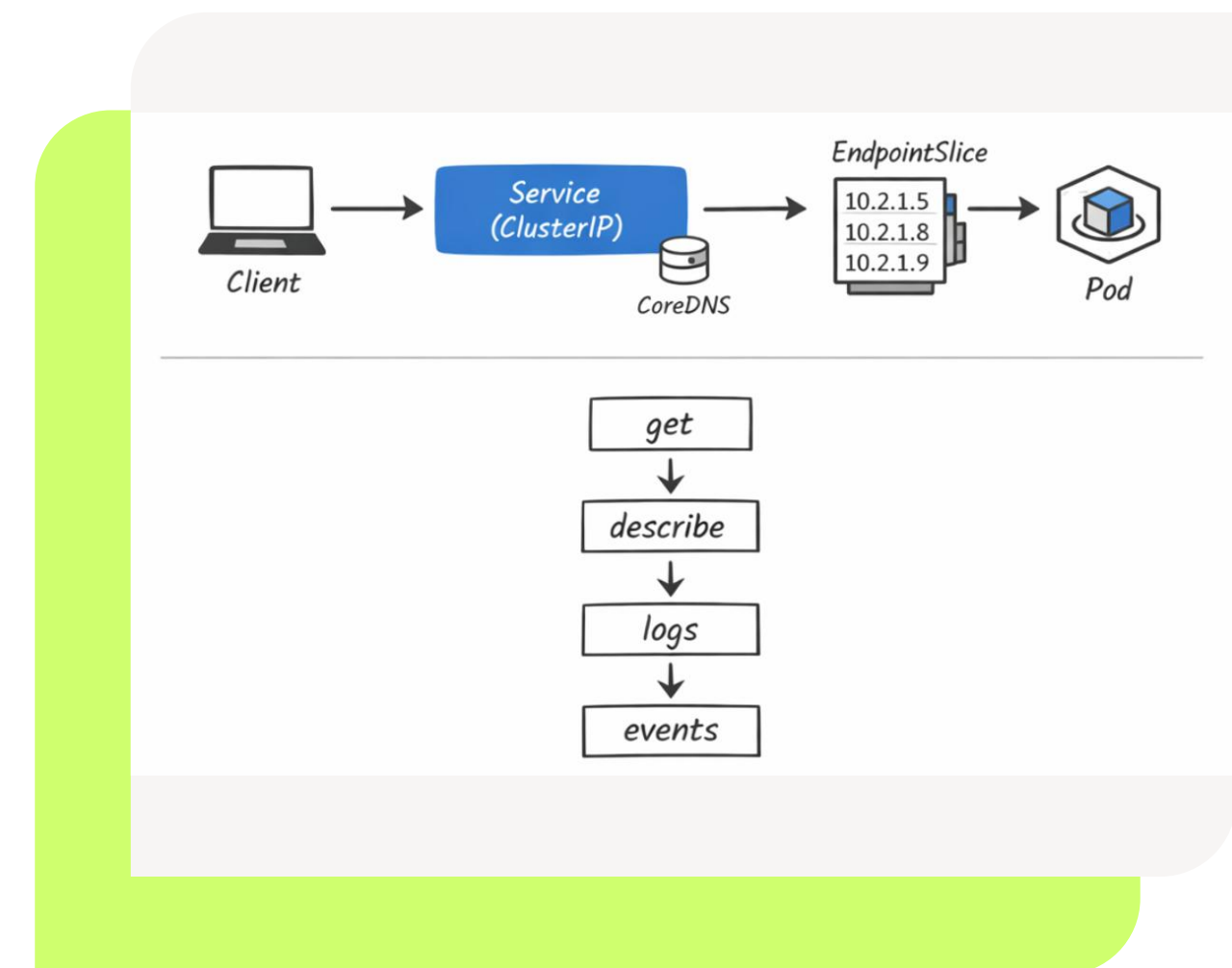
kubeadm join uses token + CA cert hash for mutual verification

CA cert hash prevents man-in-the-middle attacks during join

kubeadm token create --print-join-command regenerates an expired token

After join: copy admin.conf to ~/.kube/config for kubectl access

kubeadm init output includes the join command — copy and save it immediately



You lost the join command. Now what? (exam-speed lifesaver)

Lost the kubeadm init scrollback? Token expired (24-hour TTL)?

One command rebuilds the full join command, including the CA hash:

```
sudo kubeadm token create --print-join-command
```

Output (paste verbatim onto every worker that needs to join):

```
kubeadm join k8s.globomantics.local:6443 \  
  --token abcdef.0123456789abcdef \  
  --discovery-token-ca-cert-hash sha256:1a2b3c...<hex>
```

Inspect or audit existing tokens before regenerating:

```
sudo kubeadm token list
```

Custom TTL (e.g. 2-hour token for tighter security window):

```
sudo kubeadm token create --ttl 2h --print-join-command
```



**How do I set up kubectl access after
initializing the control plane?**



Setting up kubectl and understanding the post-init state

admin.conf lives in /etc/kubernetes/ (root-only by default – kubectl won't work yet)

Copy to ~/.kube/config and chown to your user (kubectl auth works)

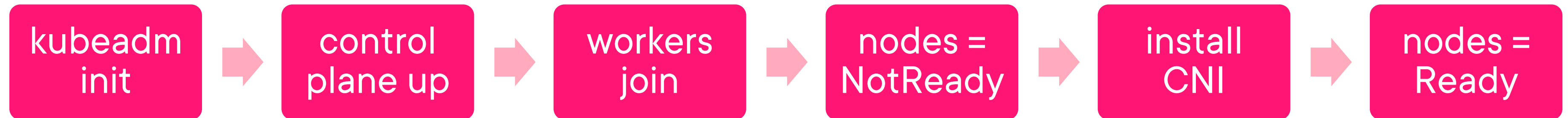
kubectl get nodes shows control plane as NotReady (expected – no CNI yet)

kube-system pods: control plane Running, CoreDNS Pending (no networking)

Static pod manifests in /etc/kubernetes/
manifests drive the control plane



From init to ready: the Kubernetes bootstrap flow



Demo

Bootstrap a cluster with kubeadm init and join two worker nodes





“Three nodes in the cluster, all showing NotReady. That is exactly where we should be. The control plane is healthy, certificates are in place, and both workers joined on the first attempt. One kubectl apply away from a running cluster.”





From Globomantics to you



pod-network-cidr must match your CNI

Calico expects 192.168.0.0/16 -- a mismatch breaks cross-node pod communication silently



kubeadm init output is your roadmap

Certificates, kubeconfig, and the join command are all in that single output — copy it immediately



NotReady after init is expected, not a failure

Nodes stay NotReady and CoreDNS stays Pending until you install a CNI plugin



All nodes must use identical versions

kubeadm, kubelet, and kubectl must match across nodes — mismatches cause join failures

